

12^h June, Group Work – through Webex

For PART B: Programming Question, you are allowed to discuss and work in groups to find the best solution for the Java coding task.

Important Submission Guidelines:

1. After your group has completed the Java programming solution, each student must write the final answer by hand (not typed).
 2. Make sure your handwriting is clear and legible.
 3. Write your name, matric number, and group members' names at the top of your answer sheet.
 4. Submit the handwritten answer in PDF or scanned image format through E-learning under the designated submission section for PART B. Individually .
-

PART B: PROGRAMMING QUESTION

Write a complete Java program that represents the UML class diagram in Figure 1 and produces the expected output depicted in Figure 2. Implement all the classes (excluding the **Order** and **FoodOrderingSystem** classes) with the specified instance variables (attributes) and methods as outlined in the diagram. The purpose of each method is as the name implies, and some of them are further explained below. Note that the definitions of the **Order** and **FoodOrderingSystem** classes are fully given in Figure 3.

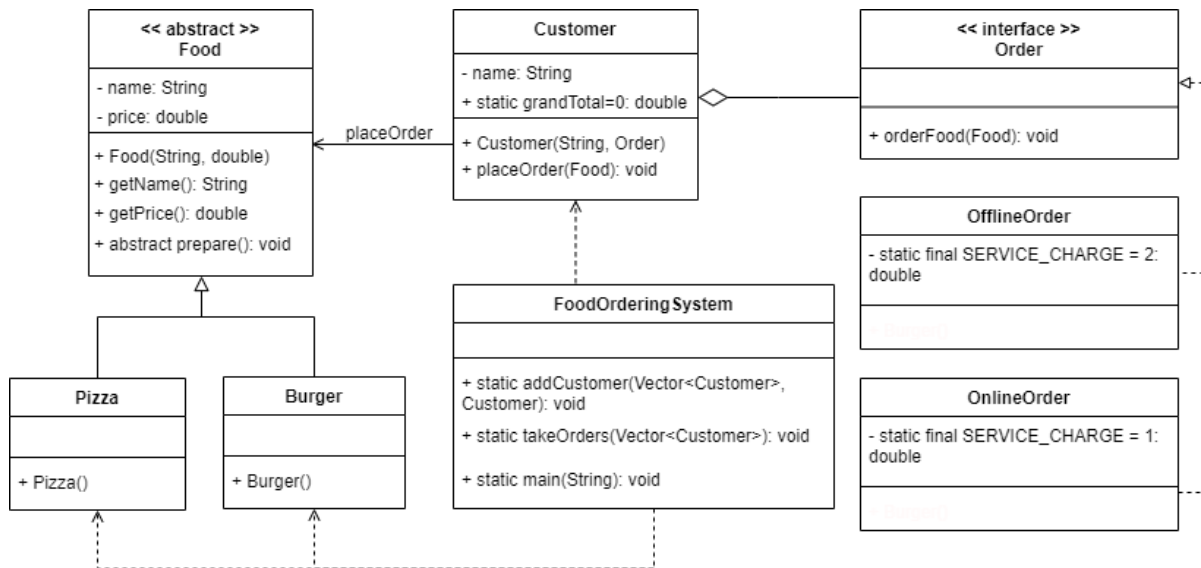


Figure 1: UML Class Diagram

```
Preparing Hawaiian Chicken...
Hawaiian Chicken ordered online.
Total: RM41.90

Preparing Double Cheeseburger...
Double Cheeseburger ordered online.
Total: RM19.35

Preparing Hawaiian Chicken...
Hawaiian Chicken ordered offline.
Total: RM42.90

Preparing Double Cheeseburger...
Double Cheeseburger ordered offline.
Total: RM20.35

Grand Total Sales = RM124.50
```

Figure 2: Expected output of the program

You are required to do the following tasks:

- a) Define a class named **OnlineOrder**: (5 marks)
- This class should implement the **Order** interface.
 - Declare a constant (SERVICE_CHARGE) representing the service charge for online orders.
 - Define an `orderFood()` method that prepares the food, calculates the cost for online order food by adding the price of food with the service charge, which is RM1.00 for online orders and updates the grand total sales. Then, display the order and the cost of the order on the screen (refer to Figure 2).
- b) Define a class named **OfflineOrder**: (5 marks)
- This class should implement the **Order** interface.
 - Declare a constant (SERVICE_CHARGE) representing the service charge for offline orders.
 - Define an `orderFood()` method that prepares the food, calculates the cost for offline order food by adding the price of food with the service charge, which is RM2.00 for offline orders and updates the grand total sales. Then, display the order and the cost of the order on the screen (refer to Figure 2).
- c) Define an abstract class named **Food**: (6 marks)
- Define a constructor that takes arguments and initializes the member attributes with passed arguments.
 - Define `getName()` method that returns the food name.
 - Define `getPrice()` method that returns the food price.
 - Declare an abstract method named `prepare()`. Subclasses must define this method.
- d) Define a class named **Pizza**: (4 marks)
- This class should inherit from the **Food** class.
 - Define a constructor and initializes the superclass's attributes with "Hawaiian Chicken" as name and RM40.90 as price.
 - Define `prepare()` method that displays pizza preparing message.
- e) Define a class named **Burger**: (4 marks)
- This class should inherit from the **Food** class.
 - Define a constructor and initializes the superclass's attributes with "Double Cheeseburger" as name and RM18.35 as price.
 - Define `prepare()` method that displays burger preparing message.

f) Define a class named **Customer**:

(6 marks)

- Define a constructor that takes arguments and initializes the member attributes with passed arguments.
- Define `placeOrder()` method that places the food order made by the customer.

```
1 // FoodOrderingSystem.java
2 import java.util.*;
3
4 interface Order {
5     void orderFood(Food food);
6 }
7
8 class FoodOrderingSystem {
9     //Method to add a customer to the system
10    public static void addCustomer(Vector<Customer> cList, Customer c) {
11        cList.add(c);
12    }
13
14    // Method to take orders from all customers in the system
15    public static void takeOrders(Vector<Customer> cList) {
16        for (Customer c: cList) {
17            c.placeOrder(new Pizza());
18            c.placeOrder(new Burger());
19        }
20    }
21
22    public static void main(String[] args) {
23        // Create a new customer list
24        Vector<Customer> cList = new Vector<>();
25
26        //Add customers to the system
27        addCustomer(cList, new Customer("Haris", new OnlineOrder()));
28        addCustomer(cList, new Customer("Aliya", new OfflineOrder()));
29
30        //Take orders from all customers
31        takeOrders(cList);
32        System.out.printf("Grand Total Sales = RM%.2f\n",
33            Customer.grandTotal);
34    }
35 }
```

Figure 3: Order and FoodOrderingSystem class definitions